

Introduction

Any software product, upon gaining its first users, faces deployment issues, which are primarily addressed by deploying multiple environments. Deployment architectures vary in each specific case, but generally, environments start with development (DEV) and end with production (PROD), with intermediate stages in between.

Using Apache Airflow, workflows are created in the form of directed acyclic graphs (DAGs) of tasks, which are convenient for monitoring and managing execution progress.

This article will focus on integrating and supporting Airflow across multiple environments, providing an example of how we addressed this issue for Kubernetes.

Problem Description

When we initially integrated Airflow into our architecture, we decided to deploy a single web server for all environments. This decision was based on several reasons at that time:

1. During the integration of Airflow with Kubernetes, we faced significant labor costs from the DevOps team, which was already limited in resources - deploying web servers for each environment and maintaining them seemed problematic;
2. Maintaining only one active Airflow server allowed the company to save costs.

In this case, separating deployment environments could only be done at the DAG definition level. We used `Git-sync` as a deployment technique, allocating a separate repository for storing files that defined the DAGs. Initially, we used the standard approach with the context manager `with` to declare DAGs (example from the documentation):

```
import datetime

from airflow import DAG
from airflow.operators.empty import EmptyOperator

with DAG(
    dag_id="my_dag_name",
    start_date=datetime.datetime(2021, 1, 1),
    schedule="@daily",
):
    EmptyOperator(task_id="task")
```

With this DAG definition, we had to create a separate file for each DAG for each environment, resulting in `number of DAGs * number of environments` files. Soon, this approach became inconvenient, as every time we deployed changes to the next environment, we had to look at the diff between files that differed only slightly, making it easy to make mistakes.

Transition to Dynamic DAG Generation

The described problem prompted us to search for a solution, which was found in a feature of Airflow called `dynamic DAG generation` - <https://airflow.apache.org/docs/apache-airflow/stable/howto/dynamic-dag-generation.html>. Notably, this documentation page suggests a solution to the problem by setting environment variables that are then used to construct the necessary graph (example from the documentation):

```
deployment = os.environ.get("DEPLOYMENT", "PROD")
if deployment == "PROD":
    task = Operator(param="prod-param")
elif deployment == "DEV":
    task = Operator(param="dev-param")
```

However, this solution was not suitable for us due to having one web server for all environments - in this case, we needed to generate all possible DAGs for all environments. This can be assisted by a design pattern known as the `Factory Method`. We will define a `Creator` class that will establish an abstract factory method (the code presented later is stored in the following repository - <https://github.com/anatoly-bobrovsky/dynamic-airflow-dags-multiple-environments>):

```
"""DAG Factory."""

from abc import ABC, abstractmethod

from .enums import EnvironmentName

class CreatorDAG(ABC):
    """Abstract DAG creator class."""

    def __init__(self, environment: EnvironmentName):
        """Initialize the creator.

        Args:
            environment (EnvironmentName): The environment name
        """
        self.environment = environment
```

```

@abstractmethod
def create(self):
    """Abstract create method."""
    pass

```

Here, `EnvironmentName` unambiguously defines the names of the deployment environments. For example, let's take two: DEV and PROD (in industrial development, there will certainly be more environments):

```

"""Enums."""

from enum import Enum

class EnvironmentName(Enum):
    """Environment name."""

    PROD: str = "prod"
    DEV: str = "dev"

```

Now we will write the necessary subclasses to create specific instances of a DAG. Suppose we need a DAG containing two tasks, one of which will be executed in a Kubernetes cluster. We will create a DAG for each environment:

```

"""DAG for test task."""

from datetime import datetime, timezone

from airflow.decorators import dag, task
from airflow.models import TaskInstance, Variable
from airflow.operators.python import get_current_context
from airflow.providers.cncf.kubernetes.operators.pod import KubernetesPodOperator
from kubernetes import client

from tools import DEFAULT_IMAGE_PULL_POLICY, IMAGE_ENVS, STARTUP_TIMEOUT,
CreatorDAG, EnvironmentName

class TestCreator(CreatorDAG):
    """Test Creator."""

    def __init__(self, environment: EnvironmentName):

```

```

"""Initialize the creator.

Args:
    environment (EnvironmentName): The environment name
"""
super().__init__(environment)
self.tags = ["test"]
self.dag_id = "test-prod" if self.environment ==
EnvironmentName.PROD else "test-dev"
self.description = "The test workflow"

def create(self):
    """Create DAG for the test workflow."""
    @dag(
        dag_id=self.dag_id,
        description=self.description,
        schedule=None,
        start_date=datetime(year=2024, month=9, day=22,
tzinfo=timezone.utc),
        catchup=False,
        default_args={
            "owner": "airflow",
            "retries": 0,
        },
        tags=self.tags,
    )
    def test_dag_generator(
        image: str = Variable.get(
            "test_image_prod" if self.environment ==
EnvironmentName.PROD else "test_image_dev"
        ),
        input_param: str = "example",
    ):
        """Generate a DAG for test workflow.

        Args:
            image (str): The image to be used for the
KubernetesPodOperator.
            input_param (str): The input parameter.
        """
        test_operator = KubernetesPodOperator(
            task_id="test-task",
            image=image,
            namespace="airflow",
            name="test-pod-prod" if self.environment ==
EnvironmentName.PROD else "test-pod-dev",

```

```

env_vars=IMAGE_ENVS,
cmds=[
    "python",
    "main.py",
    "--input_param",
    "{{ params.input_param }}",
],
in_cluster=True,
is_delete_operator_pod=True,
get_logs=True,
startup_timeout_seconds=STARTUP_TIMEOUT,
image_pull_policy=DEFAULT_IMAGE_PULL_POLICY,
do_xcom_push=True,
pool="PROD" if self.environment == EnvironmentName.PROD else
"DEV",

    container_resources=client.V1ResourceRequirements(
        requests={"cpu": "1000m", "memory": "2G"},
        limits={"cpu": "2000m", "memory": "8G"},
    ),
)

@task(task_id="print-task")
def print_result(task_id: str) -> None:
    """Print result."""
    context = get_current_context()
    ti: TaskInstance = context["ti"]
    result = ti.xcom_pull(task_ids=task_id, key="return_value")
    print(f"Result: {result}")

print_result_operator = print_result("test-task")

test_operator >> print_result_operator

return test_dag_generator()

# create DAGs for each environment
test_prod_dag = TestCreator(
    environment=EnvironmentName.PROD,
).create()

test_dev_dag = TestCreator(
    environment=EnvironmentName.DEV,
).create()

```

Let's highlight the main implementation points:

1. Import from the `tools` package - in addition to the already defined `CreatorDAG` and `EnvironmentName`, we also import:
 - Two constants: `DEFAULT_IMAGE_PULL_POLICY` and `STARTUP_TIMEOUT`.
`DEFAULT_IMAGE_PULL_POLICY` defines the image pulling policy from the repository, while `STARTUP_TIMEOUT` sets the time within which the cluster must allocate resources and start the task; otherwise, it will fail with an error;
 - Environment variables that we want to pass into the container - `IMAGE_ENVS`;
2. In the class constructor, we define parameters for the `@dag` decorator. The most important of these is the `dag_id`, which serves as a unique identifier that allows us to manage the execution of a specific DAG through the web server's API;
3. In the factory method implementation, we declare a decorated function for creating the DAG and then return the result of the execution;
4. In the function, when declaring the input parameter `image`, we use Airflow Variables to fetch the required value by key. The key is determined by the environment for which the DAG is defined. This method controls the current image for each environment through variable management on the Airflow side;
5. Next, we define the two operators that will make up our DAG. The last one is shown to demonstrate the use of the previous task's result, while the first operator contains several important points:
 - To simplify monitoring, we assign a name to the created pod based on the environment;
 - We define a task pool in which the task will execute. This separates the cluster's resources by environment;
 - We won't elaborate on the other parameters of the `KubernetesPodOperator` constructor, as they are determined by the specific Kubernetes cluster configuration;
6. Finally, we create a DAG for each environment using the class defined above. It is important to note that variable names must be unique across ALL DAGs, as Airflow looks at the global scope when parsing the repository.

Thus, this solution scales to any number of DAGs, and further maintenance is simplified by the `environment` attribute, which allows us to distinguish functionality deployed in different environments.